

CAFE BILLING SYSTEM

A Python program for cafe menu, stock and bills

PROJECT REPORT	SUBMISSION DETAILS
Submitted by	SUBHANSU KUMAR
Registration number	26BCE10452
Course	Introduction to Problem Solving and Programming
Course code	CSE1021
Institution	VIT BHOPAL UNIVERSITY
Source file	main.py

1. Introduction

ABOUT THE PROJECT

This report is about a small Cafe Billing System made with Python. It runs in a terminal. It can show and update the menu, track stock, make bills, save them and find an old bill. The program uses ideas from class such as variables, if statements, loops, lists, dictionaries, functions and files.

- The program is in one file called main.py.
- The menu and stock are saved in cafe_data.json. Bills are saved in bills.txt.
- The test ideas in this report have not been run yet.

2. Problem Statement

THE PROBLEM

A small cafe needs a simple way to show its menu, take an order, work out the bill and keep stock up to date. Adding prices manually can lead to mistakes. A shop may also accept an order when it does not have enough stock. Old bills can be hard to find if they are not saved.

- This program offers a simple way to do these tasks on a computer.
- It checks that the order does not ask for more than the stock available.
- It works out the bill and saves a copy for later.

3. Project Objectives

WHAT THE PROJECT SHOULD DO

- Show menu items, prices and stock.
- Add a menu item by entering its name, price and starting stock.
- Change the stock for an item.
- Take an order and make a bill.
- Calculate SGST, CGST and the final amount.
- Save the menu, stock and bills between runs.
- Find a saved bill using its bill number.

4. Scope and Target Users

WHO CAN USE IT

This program is made for one person using a local computer. It is a course project, not a full shop billing system.

- A small cafe owner or its staff.
- Can show and change the menu, take orders, make bills and find old bills.
- Does not have a login, a screen with buttons, online payments or a database server.

5. Functional Requirements

MENU, STOCK AND BILLING

The program has seven main features. Each item below describes one feature and how the code handles it.

5.1 Show menu

Show item number, name, price and stock. Load saved menu data or use the four default items.

5.2 Add menu item

Check that the name is not blank. Price must be a whole number above zero; stock must be zero or more. Give it the next item number and save it.

5.3 Change stock

Choose an item that is on the menu. Enter a whole-number stock count of zero or more, then save the change.

5.4 Make a bill

Add one or more menu items. Each quantity must be a whole number above zero and within stock. Enter G to finish; an empty order makes no bill.

5.5 Work out the total

Item cost = price x quantity. Add item costs for the subtotal. SGST and CGST are each 2.5% of the subtotal. Add both taxes and show two decimal places.

5.6 Save menu and bills

Save menu and stock in cafe_data.json. Add each bill to bills.txt so it stays after the program closes.

5.7 Find a saved bill

Search the bill file using the bill number. Show the matching bill or a message if the file or bill is not found.

6. Input and Output

WHAT GOES IN AND WHAT COMES OUT

- The user types a menu choice, item number, name, price, stock, order quantity or bill number.
- The program prints the menu, messages, order details and bill total.
- `cafe_data.json` stores menu names, prices and stock.
- `bills.txt` stores bill numbers, dates, items, taxes and totals.

7. How the Program Runs

USER STEPS

The program shows its menu and waits for a choice. It does the chosen task, then shows the menu again. It keeps doing this until the user chooses option 6. To make a bill, the user enters G after adding all items.

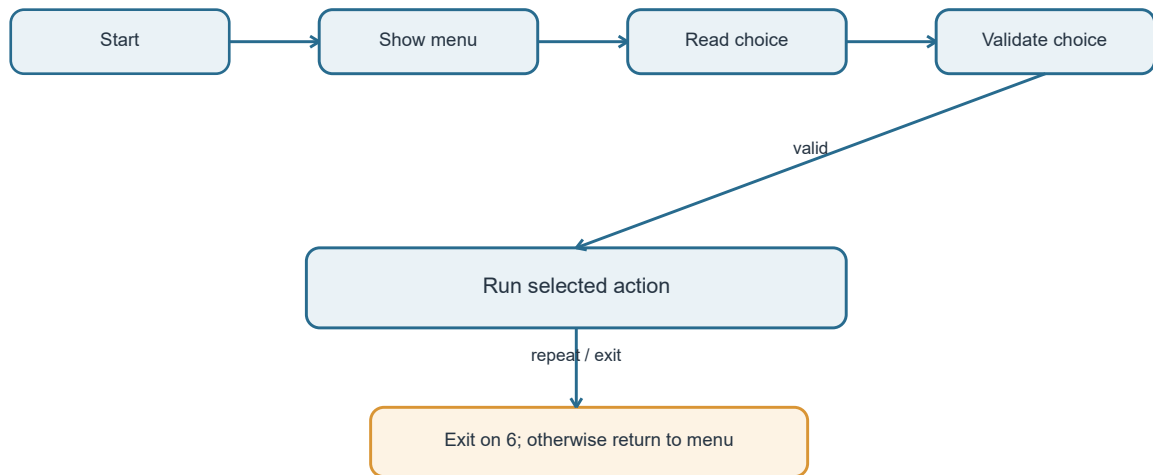


Figure 1. The menu comes back after each task until the user exits.

8. Non-Functional Requirements

HOW THE PROGRAM SHOULD WORK

These points describe ease of use, safety, speed and data handling in the supplied code.

8.1 Easy to use

A numbered menu, short prompts and clear messages guide the operator.

8.2 Reliable results

Checks inputs and stock before accepting changes. Damaged files may still stop the program.

8.3 Handle errors

Shows messages for wrong choices, bad numbers, unknown items and missing bills. Not every file problem is handled.

8.4 Speed

Works with menu and order data in memory. Bill search reads the full text file.

8.5 Easy to maintain

Functions separate the tasks, but all code is in main.py. The guideline asks for 5-10 useful files or modules.

8.6 Use few resources

Uses Python tools json, os and datetime. No extra package or server is needed.

8.7 Save data

Menu and stock are saved in JSON; bills are added to a text file. There are no backup copies.

8.8 Security

There is no login, password or encryption. Anyone with access to the folder can read or change its files.

9. System Architecture

HOW THE PARTS CONNECT

The main.py file controls the program. It reads what the user types, runs the task, shows the result and reads or saves local files. All program functions are in the same file.

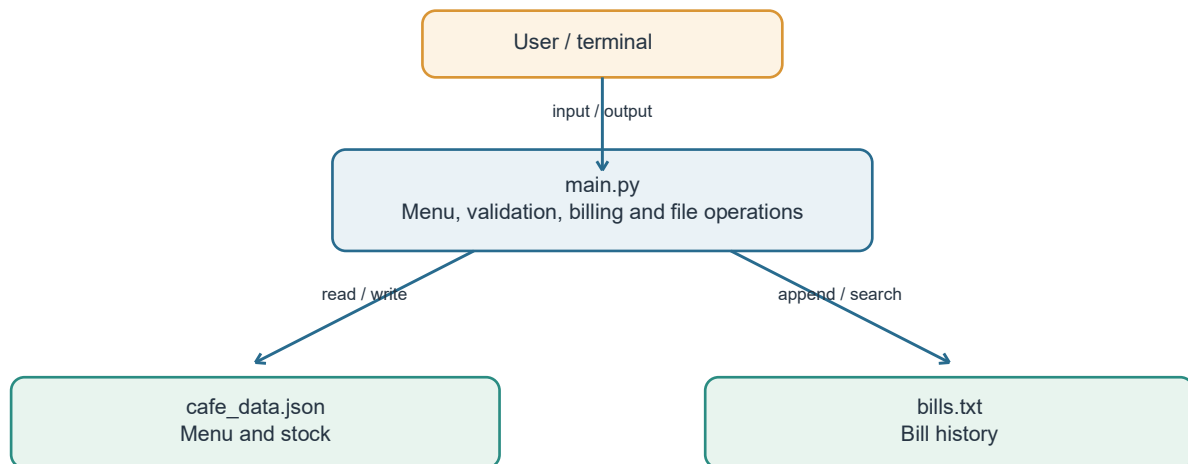


Figure 2. Main program and the two files it uses.

10.1 Design Diagram: Use Case

WHAT THE USER CAN DO

The user can show the menu, add an item, change stock, make a bill, find a saved bill or exit.

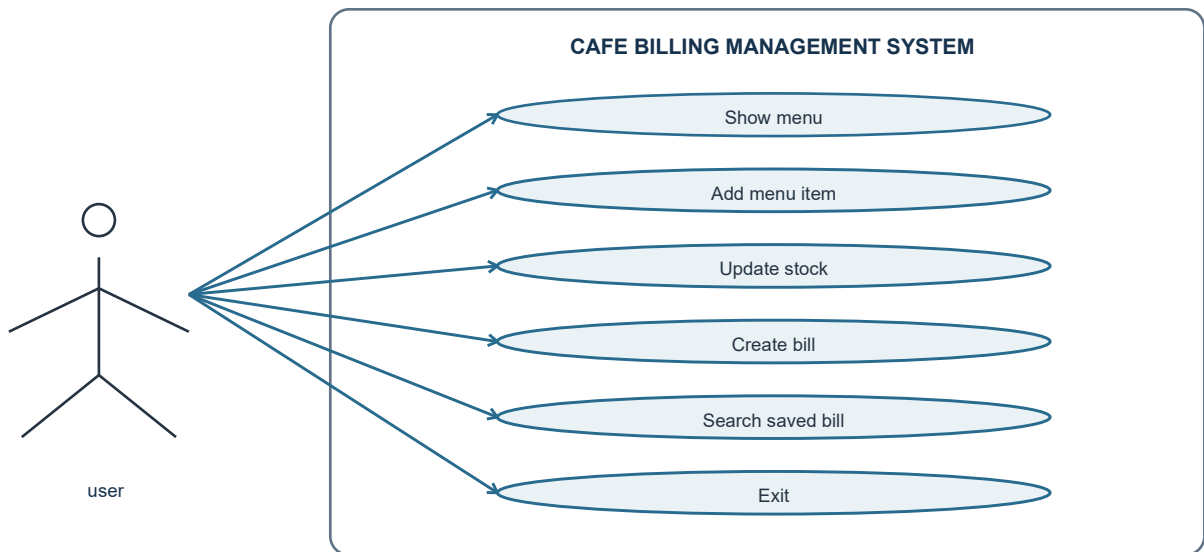


Figure 3. Actions available from the program menu.

10.2 Design Diagram: Workflow

STEPS THROUGH THE PROGRAM

The program repeats the main menu until the user exits. While making a bill, it keeps asking for items until the operator enters G.

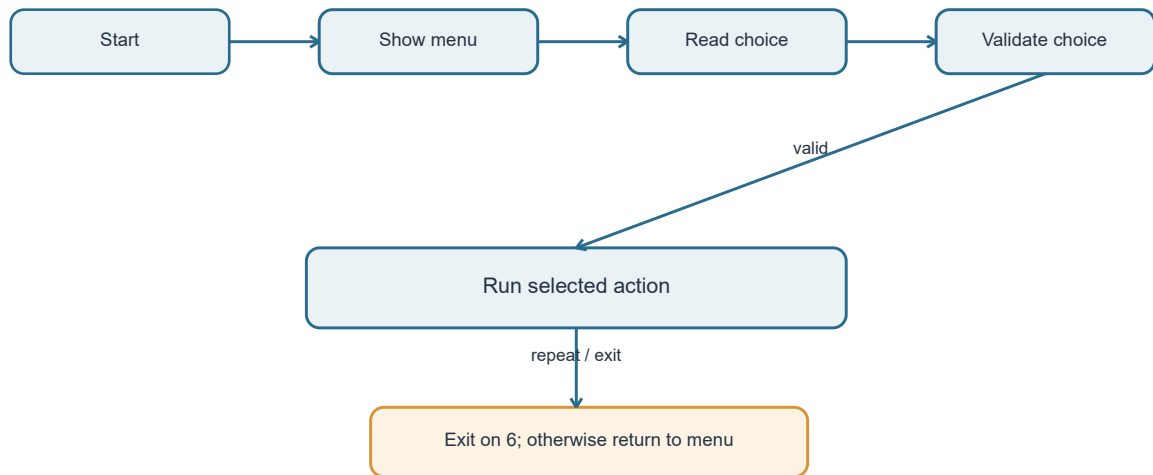


Figure 4. Main menu and action flow.

10.3 Design Diagram: Bill Steps

STEPS WHEN MAKING A BILL

The program reads the menu, takes the order, saves the new stock, saves the bill and shows the total.

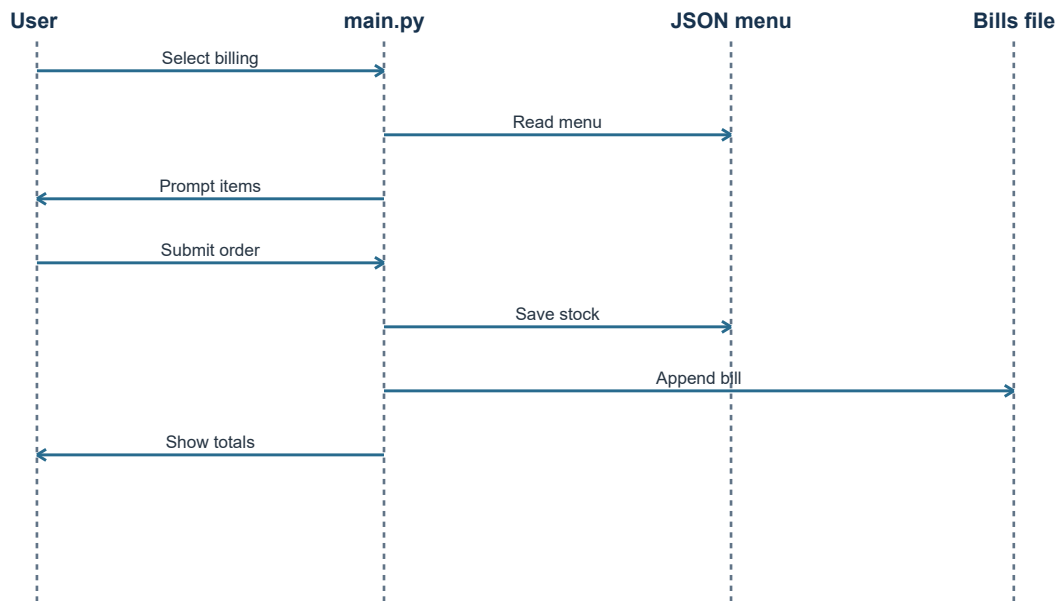


Figure 5. Main steps used to make and save a bill.

10.4 Design Diagram: Program Parts

FUNCTIONS IN THE CODE

The code does not use classes or separate files. This diagram groups its functions by what they do.

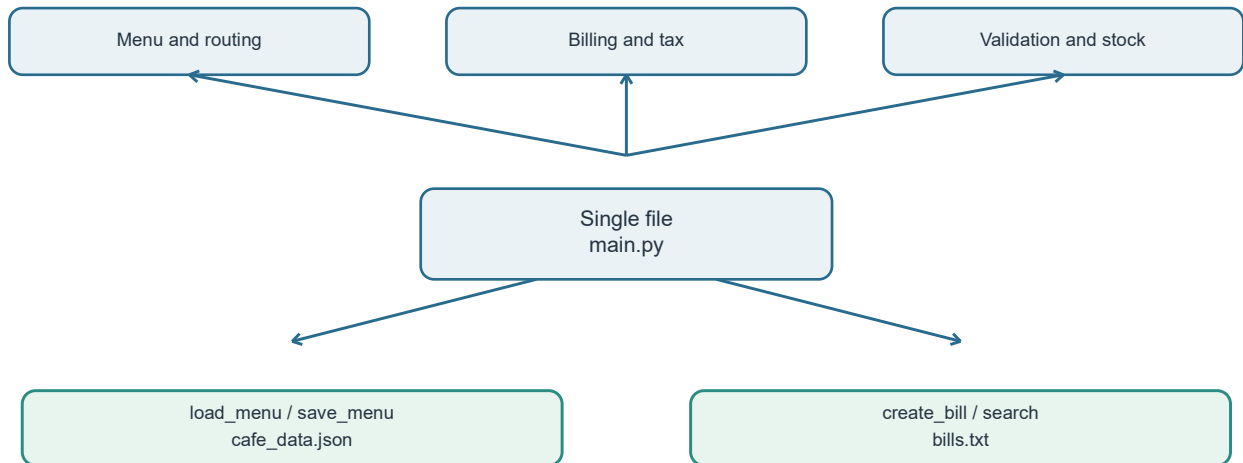


Figure 6. These parts are all inside main.py.

10.5 How Data Is Saved

FILES USED BY THE PROGRAM

The program uses two local files. It does not use a database, so a database diagram is not needed.

File	What it contains	Functions
cafe_data.json	Menu item number, name, price and stock.	load_menu(), save_menu()
bills.txt	Bill number, date, items and totals.	create_bill(), show_saved_bill()

11. Design Choices

WHY THESE CHOICES WERE MADE

- Dictionaries keep menu details together under clear names.
- Lists hold the items in one order.
- JSON keeps menu data in a simple format.
- A text file makes saved bills easy to read.
- The program checks earlier items in an order before accepting more of the same item.
- SGST and CGST are shown as two separate amounts.

Bill numbers can repeat if two bills are made in the same second. Bill search may also match part of a number.

12. How the Code Works

PROGRAM DETAILS

The program is written in Python 3. It uses built-in tools for JSON files, checking whether files exist and getting the date and time. The main menu calls functions in main.py.

Function	What it does
load_menu()	Loads the saved menu or uses the starting menu.
save_menu()	Saves menu items and stock.
show_menu()	Shows the menu.
add_item()	Checks and adds a menu item.
update_stock()	Checks and changes stock.
create_bill()	Takes an order, works out totals and saves the bill.
show_saved_bill()	Finds and shows a saved bill.

13. Example Bill

EXAMPLE RESULT

10 Teas cost each Rs. 15, 5 Sandwich cost each Rs. 60 and 5 Cakes cost each Rs. 40. SGST is Rs. 16.25 and CGST is Rs. 16.25. The final total is Rs.682.5

```
===== CAFE BILL =====  
Bill number: B20260930171334  
Date: 30-09-2026 17:13  
-----  
Tea x 10 = Rs. 150  
Sandwich x 5 = Rs. 300  
Cake x 5 = Rs. 200  
-----  
Subtotal: Rs. 650  
SGST: 2.5 % = Rs. 16.25  
CGST: 2.5 % = Rs. 16.25  
Grand total: Rs. 682.5  
=====  
Bill saved in system  
Updated stock saved in menu
```

14. Testing Approach

CHECKS TO TRY

These checks have not been run yet.

#	What to try	What should happen	Status
1	Start with no saved menu	Show four starting items	Not run
2	Add a valid item	Add and save it	Not run
3	Enter a blank name or bad price	Reject the entry	Not run
4	Change stock with bad input	Reject the change	Not run
5	Make a valid bill	Show and save the total	Not run
6	Order more than stock	Reject the order	Not run
7	Enter a bad quantity	Reject the quantity	Not run
8	Restart after a bill	Load saved stock and bill	Not run
9	Search for a saved bill	Show that bill	Not run
10	Use a wrong choice or bill number	Show a message	Not run

15. Challenges in the Program

THINGS TO HANDLE CAREFULLY

- Each kind of input needs its own check.
- The stock check must count items already added to the same order.
- The bill must show each cost, both taxes and the final amount.
- Saved files should be checked again after closing and reopening the program.

16. What I Learned

MAIN LESSONS

- Functions help split a large task into smaller jobs.
- Lists and dictionaries are useful for storing orders and menu items.
- Checking entries early helps avoid mistakes.
- JSON and text files can keep information after a program closes.
- Tests should check bad entries and saved data, not only a successful bill.

17. Future Improvements

WHAT CAN BE ADDED LATER

- Add tests and take real screenshots of the program.
- Show a helpful message if a saved file is damaged.
- Make bill numbers unique and search the exact bill number.
- Add bill lists, date searches and daily sales totals.

18. References

SOURCES USED

- VITyarthi, Build Your Own Project instructions
- Python 3 guide
- Python guides for json and datetime.
- The café billing management system code which is in main.py.

19. Screenshots

MENU AND ORDER ACTIONS

```
--- CAFE BILLING SYSTEM ---
1. Create a new bill
2. Search for a bill
3. Add a menu item
4. Show menu
5. Update stock
6. Exit
Choose an option: 4

--- CAFE MENU ---
1 - Tea | Price: Rs. 15 | Stock: 10
2 - Coffee | Price: Rs. 30 | Stock: 15
3 - Sandwich | Price: Rs. 60 | Stock: 5
4 - Cake | Price: Rs. 40 | Stock: 3
```

19.1 Menu

```
--- CAFE BILLING SYSTEM ---
1. Create a new bill
2. Search for a bill
3. Add a menu item
4. Show menu
5. Update stock
6. Exit
Choose an option: 5

--- CAFE MENU ---
1 - Tea | Price: Rs. 15 | Stock: 10
2 - Coffee | Price: Rs. 30 | Stock: 15
3 - Sandwich | Price: Rs. 60 | Stock: 5
4 - Cake | Price: Rs. 40 | Stock: 3
5 - chicken | Price: Rs. 100 | Stock: 20

Enter the item number to update: 1
Enter the new stock quantity: 20
Tea stock updated to 20

--- CAFE BILLING SYSTEM ---
1. Create a new bill
2. Search for a bill
3. Add a menu item
4. Show menu
5. Update stock
6. Exit
Choose an option: 4

--- CAFE MENU ---
1 - Tea | Price: Rs. 15 | Stock: 20
2 - Coffee | Price: Rs. 30 | Stock: 15
```

19.2 Stock update

19. Screenshots

ADD ITEM AND BILL RECORDS

```

--- CAFE BILLING SYSTEM ---
1. Create a new bill
2. Search for a bill
3. Add a menu item
4. Show menu
5. Update stock
6. Exit
Choose an option: 3

--- ADD A MENU ITEM ---
Enter item name: chicken
Enter item price: 100
Enter stock quantity: 20
chicken has been added and saved.

--- CAFE BILLING SYSTEM ---
1. Create a new bill
2. Search for a bill
3. Add a menu item
4. Show menu
5. Update stock
6. Exit
Choose an option: 4

--- CAFE MENU ---
1 - Tea | Price: Rs. 15 | Stock: 10
2 - Coffee | Price: Rs. 30 | Stock: 15
3 - Sandwich | Price: Rs. 60 | Stock: 5
4 - Cake | Price: Rs. 40 | Stock: 3
5 - chicken | Price: Rs. 100 | Stock: 20

```

19.3 Add menu item

```

===== CAFE BILL =====
Bill number: B20260930175851
Date: 30-09-2026 17:58
-----
Tea x 5 = Rs. 75
chicken x 10 = Rs. 1000
-----
Subtotal: Rs. 1075
SGST: 2.5 % = Rs. 26.88
CGST: 2.5 % = Rs. 26.88
Grand total: Rs. 1128.75
=====
Bill saved in system
Updated stock saved in menu

```

19.4 Generated bill

```

--- CAFE BILLING SYSTEM ---
1. Create a new bill
2. Search for a bill
3. Add a menu item
4. Show menu
5. Update stock
6. Exit
Choose an option: 2
Enter bill number: B20260930175851

--- BILL FOUND ---

Bill number: B20260930175851
Date: 30-09-2026 17:58
Tea x 5 = Rs. 75
chicken x 10 = Rs. 1000
Subtotal: Rs. 1075
SGST: Rs. 26.88
CGST: Rs. 26.88
Grand total: Rs. 1128.75

```

19.5 Search saved bill